

If I have to design Access Rights page now... where Super Admin will have hard coded entry in database and full rights of entire application and to control everything.

We have almost 12+ pages as of now and in future this can increase as well.
I wish to design one Access Rights page for granting employee wise rights on pages.

Navigation Placement

Add this page under the existing SETUP section.

The page must be accessible only to users having the Super Admin role.

Do not show this menu item to any other user.

Permission Scope

Permissions are required only at page and subpage level.

Do not create separate permissions for:

- * Add
- * Edit
- * Delete
- * Approve
- * Export
- * View Own
- * View Team
- * Any button-level action

A page permission only determines whether the employee can open and see that page in the application.

Important Separation of Access Rules

There are two separate access concepts:

1. Page Access

Page access is manually assigned by the Super Admin through this page.

It determines which menu pages and subpages the employee can access.

2. Data Visibility

Data visibility must not be manually configured on this page.

Data visibility is automatically derived from the employee hierarchy already maintained in Employee Master.

The default data visibility rule is:

- * An employee can view their own data.
- * A Resource Owner can view data of their directly assigned employees.
- * A Resource Owner can also view data of all indirect subordinate employees under the hierarchy.

Example:

- * Employee A is Resource Owner of Employee B.
- * Employee B is Resource Owner of Employee C.
- * Employee A can view data belonging to Employee A, Employee B and Employee C.
- * Employee B can view data belonging to Employee B and Employee C.
- * Employee C can view only Employee C's data.

This hierarchy must be recursive and must support any number of levels.

Do not add a manual employee-selection or data-scope configuration inside the Access Rights page.

Recommended Page Flow

Use the simplest possible employee-first flow.
The page must contain two main sections:

Left Section: Employee Selection
Provide a searchable employee list.

Display:

- * Employee name
- * Employee email ID
- * Department, if available
- * Active or inactive status
- * Resource Owner name, if available

Provide search by:

- * Employee name
- * Email ID
- * Department

By default:

- * Show active employees first.
- * Allow inactive employees to be included through a filter.
- * Do not allow access rights to be edited for an inactive employee unless the current application already supports such behaviour.

When an employee is selected, load the employee's current page rights in the right section.

Use the most appropriate existing UI pattern from the application, such as:

- * Left-side employee list with right-side configuration panel, or
- * Employee table with a Manage Access action opening a drawer

Placement & gating

- Lives under **Setup**. The menu item is **rendered only for Super Admin** — not disabled-greyed for others, absent entirely. Route is also role-guarded, so a direct URL hit by a non-admin bounces.

Left — employee selection

- Search across **name / email / department**; **Active shown first**, an "All" toggle pulls in inactive.
- Rows carry name, email, department, RO name, active/inactive chip — plus a **rights count ("9/13")** so you can eyeball who's wide-open vs locked-down at a glance.
- **Inactive employees are read-only** (Sneha, dimmed) — selectable to *view* their rights, but toggles are disabled, honoring your "don't edit inactive unless already supported" rule.

Right — page access only

- One toggle per page = *can this person open it*. No Add/Edit/Delete/Export/View-scope sub-permissions — exactly the single dimension you specified.
- **Grouped by nav section** (Planning / Reports / Dashboards / Setup) with a section-level **Toggle all** and an "N of M" count. This is the key scaling move: **new pages appear automatically under their section** — the page never needs redesigning as your 12+ grows.
- **Subpages nest** under their parent (Configuration Masters → Organization / Skills / Activities), which is where your "page and subpage level" requirement shows up.

The data-visibility separation (the important one)

- A **read-only lilac banner** states data visibility is *derived from hierarchy, not set here*, and shows the selected employee's actual reach (own + direct + indirect reports, recursive). There is deliberately **no employee-picker or data-scope control** — the banner exists precisely so the admin isn't confused by its absence.

Super Admin

- Hardcoded account: panel is **locked fully on**, no Save/Discard, with a clear note. Guarantees the controller of the app can never be locked out.

Save model — *explicit Save/Discard (shown, safer for a permissions screen).*

And the **"Copy access from..."** - *It's the one compliant shortcut (still pure page-access, no roles) that makes onboarding a new hire fast without violating your no-role-templates rule.*